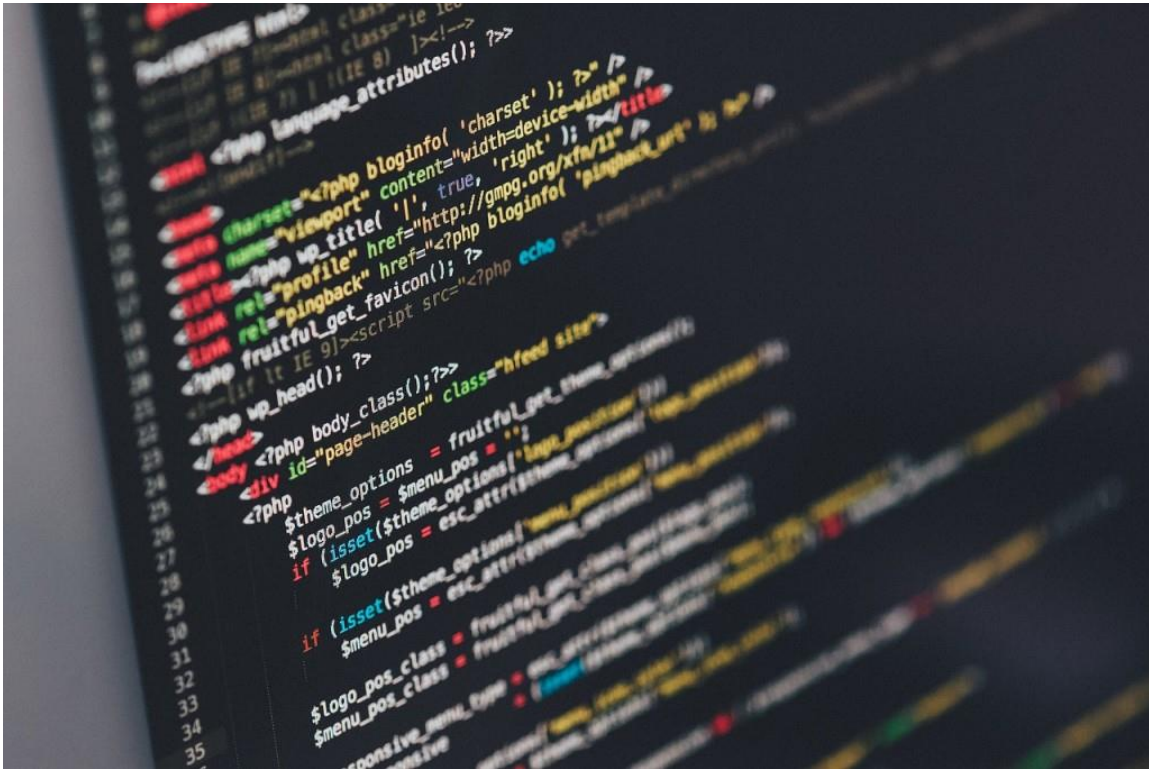


Collaborative Code Editor

Coding with real time changes and no conflicts!



Outlines:

- Introduction
- Components and Features
- Real-time communication
- Version Control System
- Security
- Clean Code
- Effective Java
- Design Patterns

Introduction

First let us consider what we will be building:

- **Collaborative Code editor:**
 - Which is a **web-based** code editor that allows the users to coding **collaboratively**, allowing them to choose either to code alone, or inviting others to code with **real time** changes.
 - For the **real-time** updates, we will use **Websockets** to ensure that the changes are delivered in the update time, but still, we will use the **Http** refreshing for some cases (when real-time is not necessary).
 - For the users, each user will be a directory on the **filesystem**, and this directory will have all the **user-related files** and folders (including the .vcs folder that we will take about soon).
 - The main purpose is of course considered, which is running codes on the web, and we will consider and support three languages to run: **Java**, **C++**, and **Python**.
 - Users will be able to add **projects**, **folders** and **files** in these projects, share these projects either to write or to just view, delete these projects, changes visibility of these projects.
 - There will be also **Admins** that we will see their role and what they can do later.
- **Version Control System:**
 - Which is simplified **Git**, that we will use to **track the changes** and almost all the git functionalities like **push**, **fork**, **revert**, **log** and so on.
 - We will use the filesystem to store the files and folders of the user's projects, and inside these projects, we will save the **.vcs** file (if the user initialized this directory, which is the version control directory), and all the related files will be stored inside this directory.
 - For security and memory complexity, we will optimize everything to ensure a light, secure version control system.
- **Code Execution:**
 - The **Java**, **C++**, and **Python** codes will be compiled using the **ProccessBuilder** and a docker container, but we will not use a docker container when running the application inside docker (containerize the application) to avoid the **DinD** (Docker in Docker) complexity.

Note: We will write the directories and the files on the filesystem.

Components and Features

Next, we will discuss each of the components and the features of the system:

- **Users:**

Each user will be either an **Editor** or an **Admin**, the Editor could be also a **Viewer** (depends on the **Role** of the user), the editor will have his/her own projects, and the admin will have no projects but will have some privileges to the creation and the deletion of the users and the projects and other features.

The user (**Client**) class will have the following attributes:

```
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;
private String name;
@Column(unique = true)
private String email;
private String password;
@Enumerated(EnumType.STRING)
private Role role;

@OneToMany(mappedBy = "client", fetch = FetchType.LAZY)
@JsonIgnore
private List<Token> tokens;

@OneToMany(mappedBy = "client", fetch = FetchType.LAZY)
@JsonIgnore
private List<Project> projects;
```

The **id** to identify each client, the name and email and password, and the **Role** (which is an enum class that we will see later), a list of **tokens**, and a list of **projects**.

- **File Item:**

Which will be the parent of the **Folder** and **File** classes, and have the following attributes:

```
private Long id;
private String name;
private Long parentId;
private Boolean isFolder;
```

Each file item will have one parent, which will introduce a tree that we will discuss later (the tree of the files and folders).

- **Folder and Snippet:**

Each one of these classes is a child of the **FileItem** class, and they will just inherit the attributes of the parent class **FileItem**, but with setting the attribute **isFolder**, for each one of them.

So, for example the **Snippet** class will have a constructor as the following:

```
public Snippet(String name, Long parentId, Long id) {  
    super.setId(id);  
    super.setName(name);  
    super.setParentId(parentId);  
    super.setIsFolder(false);  
}
```

As we can see, we set the **IsFolder** attribute to be **false** and will be **true** for the Folder class.

- **Project:**

The project will be as follows:

```
@Id  
@GeneratedValue(strategy = GenerationType.IDENTITY)  
private Long id;  
@Column(nullable = false)  
private String name;  
@ManyToOne  
@JoinColumn(name = "client_id")  
@JsonIgnore  
private Client client;
```

Each project will be associated with a client, but one client can have multiple projects.

- **File Node:**

The file node is the class that will hold the data about the file in the project structure (project tree that we will see soon), and will have the following attributes:

```
private String name;  
private List<FileItem> children;  
private Long parentId;
```

The children will represent all the subfolders and subfiles, and we will use the parent id to navigate between the files and folders by their parents, as we will see next in the project structure.

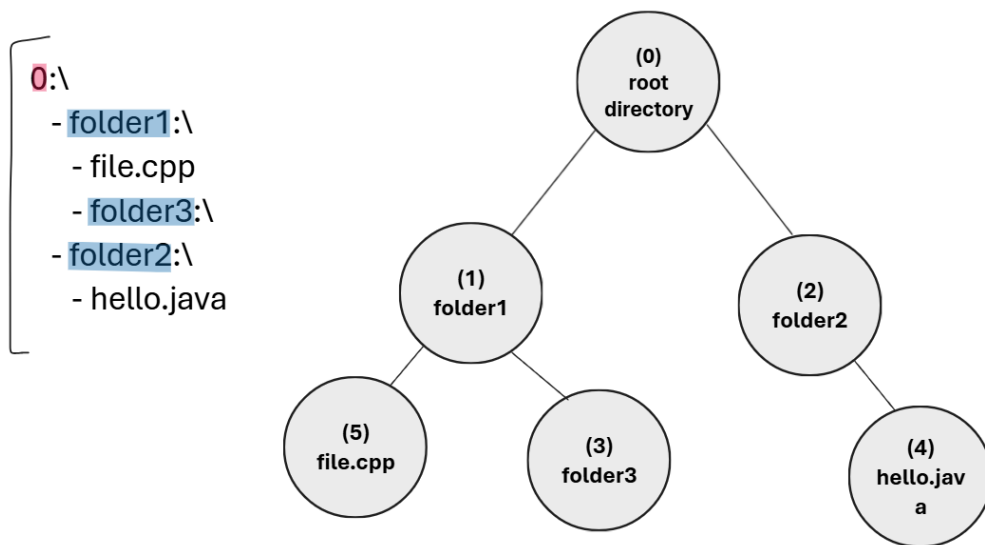
- **Project Structure:**

The structure of the project will be represented as a **Tree** using a **Hash Map**, where each key is a long number representing the id of the file item, and each value will be a file node that will represent the data about this file item as the following:

```
private Map<Long, FileNode> tree;

public ProjectStructure() { 3 usages  AhmdJabr
    tree = new HashMap<>();
    tree.put(0L, new FileNode( name: "", new ArrayList<>(), parentId: null));
}
```

Let's first understand the tree and what it contains:



This is what we want to achieve to make the structure of the project, and this will be done using the hash map, each key in this hash map will represent the current file id, and the value will be a file node (data) about the current file, so the above tree can be represented in the hash map as the following:

```
{
  "0": {
    "name": "",
    "children": [
      {
        "name": "folder1",
        "parentId": 0,
        "id": 6,
        "type": "Folder"
      },
      {
        "name": "folder2",
        "parentId": 0,
        "id": 7,
        "type": "Folder"
      }
    ],
    "parentId": null
  },
  "6": {
    "name": "folder1",
    "children": [
      {
        "name": "file.cpp",
        "parentId": 6,
        "id": 8,
        "type": "Snippet"
      },
      {
        "name": "folder3",
        "parentId": 6,
        "id": 9,
        "type": "Folder"
      }
    ],
    "parentId": 0
  },
  "7": {
    "name": "folder2",
    "children": [
      {
        "name": "hello.java",
        "parentId": 7,
        "id": 10,
        "type": "Snippet"
      }
    ],
    "parentId": 0
  },
  "9": {
    "name": "folder3",
    "children": [],
    "parentId": 6
  }
}
```

As we can see, the root directory (the parent of all the folders and files) is the directory with the id: 0, and this directory will have no name and no parent, for the other directories like **folder2** for example, the id is 7, and the name is folder2, and the children (list of all the subfiles and subfolder) are the file 10 (hello.java), and we can also see that the files (we will call them code snippets from now on) are not included as an items in this hash map, because the idea is to keep track the children of each directory.

- **Comment:**

Which will hold the data about the person who made the comment (we will see how the comment works in our case later), the comment will have the following attributes:

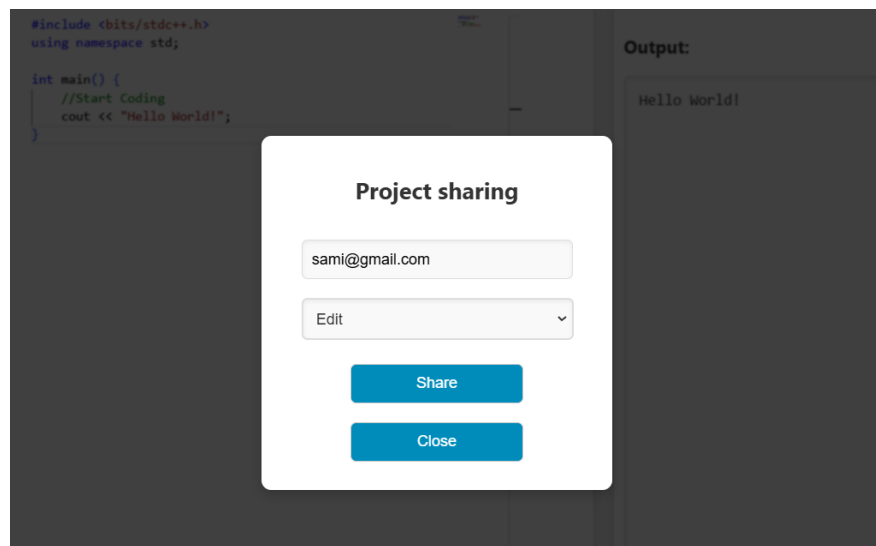
```
private String editorName;  
private String editorEmail;  
private Integer start;  
private Integer end;  
private LocalDate date;  
private String content;
```

- **Client Features:**

- **Sharing Projects:**

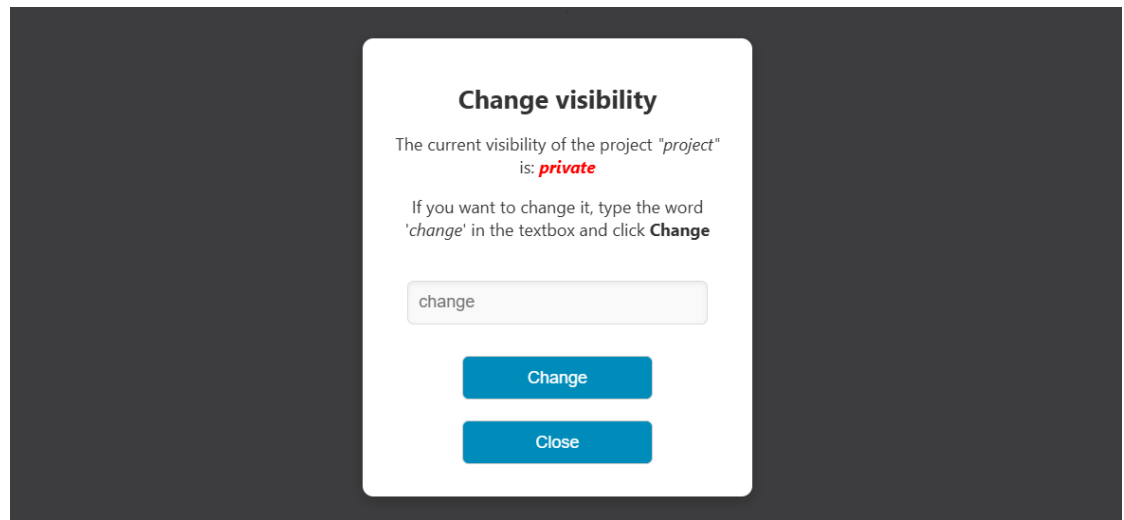
Which allows the users to share the project with other clients to edit or to view, so when sharing the project to edit, we will allow the shared user to update directly on the **project itself**, otherwise the user will not be able to write on the project.

When sharing project to edit (or **view**), the client will have this project on the “Shared to edit” section on the projects page, and so on for the share with view.



- **Sharing Projects to Public:**

Users can also share their projects to public repositories (or **undo share** it, make it private again), meaning when editors go to the public repo, and browse for the user that shared the project no public, they will see the list of the projects that have been shared on public, and editors can enter this project and fork it or just watching (**view mode**).

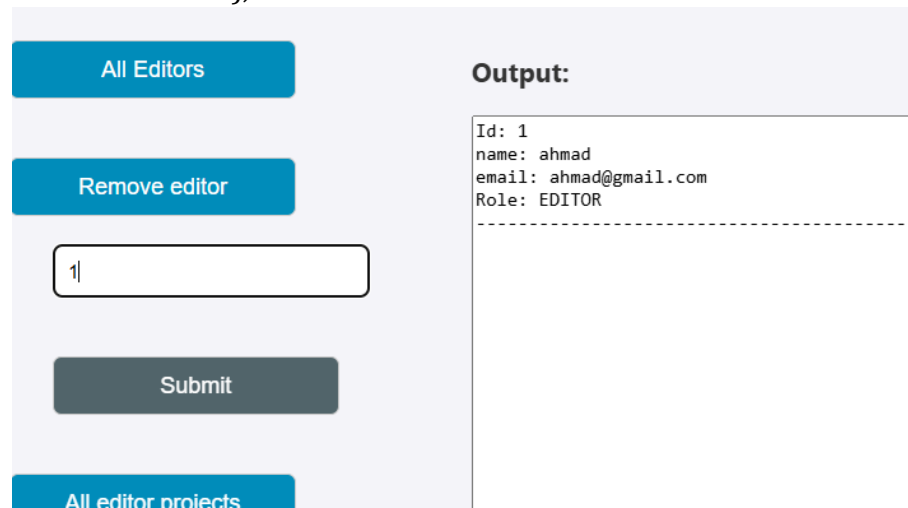


A modal dialog box titled "Change visibility" with a dark background. The text inside states: "The current visibility of the project 'project' is: **private**". It then instructs the user: "If you want to change it, type the word 'change' in the textbox and click **Change**". Below this is a text input field containing the word "change". At the bottom are two blue buttons: "Change" and "Close".

- **Admin Features:**

- **Remove Editors:**

If the admin knows the editor id (which can be **fetch**ed from the get all **editors feature**), the admin could remove this client and all its data and files.



The admin interface is divided into two main sections. On the left, there are four buttons: "All Editors", "Remove editor", "Submit", and "All editor projects". Below the "Remove editor" button is a text input field containing the number "1". On the right, under the heading "Output:", there is a box containing the following text: "Id: 1", "name: ahmad", "email: ahmad@gmail.com", and "Role: EDITOR". A dashed line is positioned below the "Role: EDITOR" text.

- **Remove Editors Projects:**

The admin also can remove any project for any editor just by knowing the project id to delete (which can be fetched by the get **all editors** projects).

All Editors

Remove editor

All editor projects

Remove project from editor

1

3

Submit

Output:

```
Id: 1
name: test
-----
Id: 2
name: testing
-----
Id: 3
name: project
-----
```

will be removed

- **Remove Shared Projects:**

Admin can also remove a shared project from a client, but still, this project will be removed if the shared strategy is **share with edit**, otherwise the admin will not be able to remove.

All Editors

Remove editor

All editor projects

Remove project from editor

All shared with

Remove shared editor

1

4

Submit

Output:

```
Id: 4
name: share with ahead
-----
```

will be removed

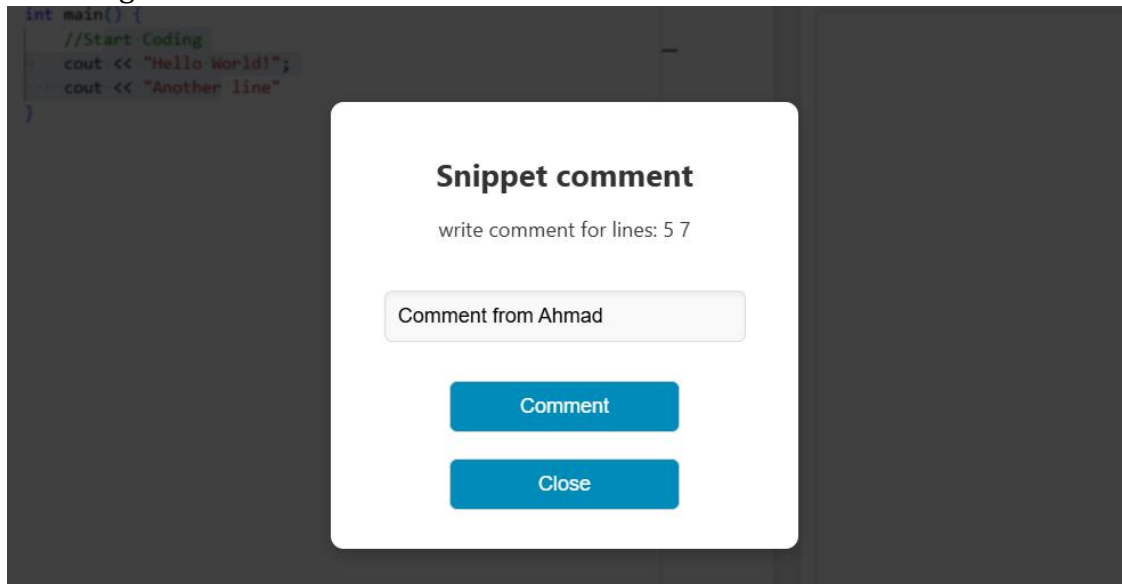
Shared with me

Edit

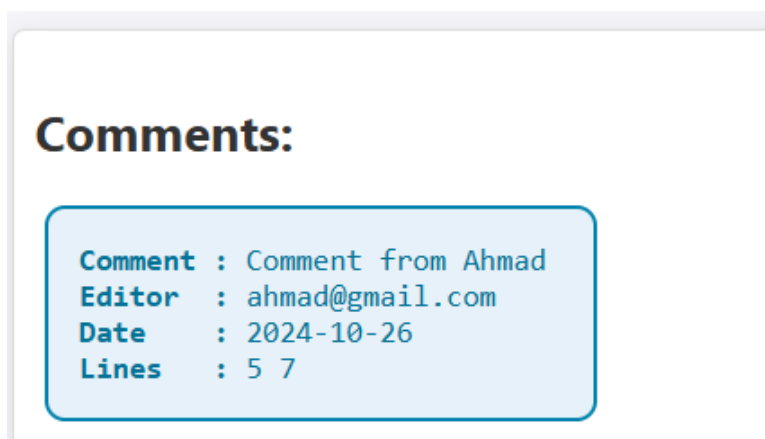
• share with ahmad

- **Comment:**

Users will be able to comment on a **specified lines of code**, when comment, this comment will be related to the snippet that the user have commented as the following:



And all the related comments to the snippet will be listed when clicking "**All Comments**"



- **Code execution:**

We can execute the snippet code (**C++**, **Java**, or **Python**) using the process builder as we said before, and when we are not containerized the application, we will use docker to isolate the execution, but when containerized the application, we will not use docker to avoid **DinD** complexity, which is allowing to use docker inside a docker container.

Real-Time Communication

- **Web Sockets:**

Which is a way for websites and apps to keep an open line between your device and the server, letting them talk to each other instantly and continuously. Unlike regular requests where you **have to** ask for new information each time, with WebSockets, either side can send updates whenever they want. This makes it perfect for things like chat apps, live sports scores, and collaborative tools, where quick, real-time updates are essential, and this is the way that we will handle **the real-time** communication in the snippets between users.

- **Web Sockets configurations:**

```
@MessageMapping("/editor/change")  @AhmdJabr *
public void handleCodeChange(Map<String, Object> body) {
    long snippetId = Long.parseLong(body.get("snippetId").toString());
    Map<String, String> resBody = new HashMap<>();
    resBody.put("change", body.get("change").toString());
    resBody.put("token", body.get("token").toString());
    messagingTemplate.convertAndSend(destination: "/topic/snippet/" + snippetId, resBody);
}
```

When client clicks on a snippet, the **web-socket** connection will be established, and the other users that are already connected to the topic with the id of the snippet will see the newly connected user.

On each update, we will send the code to the backend and just replace the code on the filesystem with the newly added code.

- **Why not Using Web Sockets on every update:**

For the updates like “create folder”, “deleting file” and so on, it is not necessary to update all the users with such updates, instead, users can just refresh the page and they will get the updates like this, and we are doing this because the web socket is heavy, and we don’t want to establish a web socket connection once the user enter the project, the connection will be open for nothing (unless two or more users start updating and seeing the same snippet).

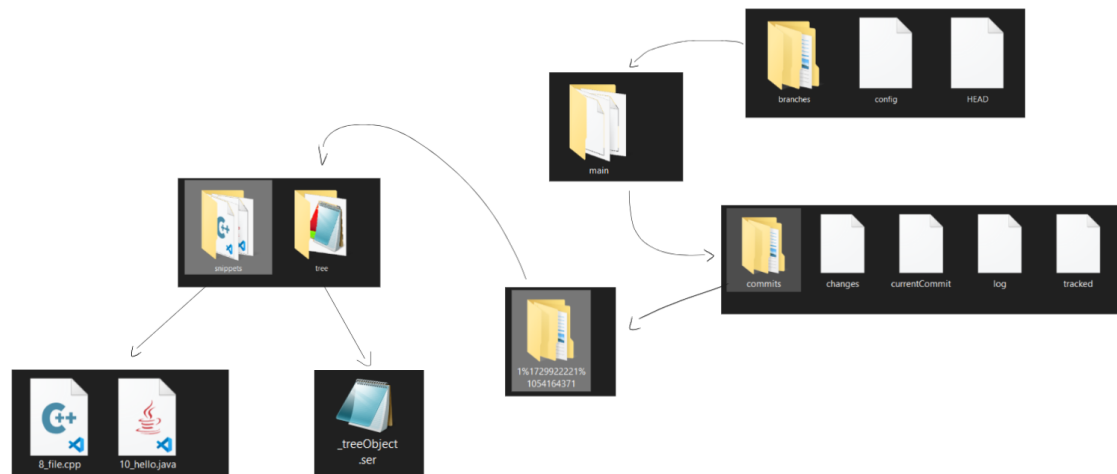
```
Received data      @stomp_stompjs.js?v=85cbd449:1214
<<< MESSAGE      @stomp_stompjs.js?v=85cbd449:1214
content-length:345
message-id:e4s42kun-6
subscription:sub-0
content-type:application/json
destination:/topic/snippet/8
content-length:345
```

Version Control System

Next, we will see the features that I've provide in the vcs:

- **Initialize .vcs directory:**

Which will add the following files and folders:



And we will talk about each one of the files and folders soon.

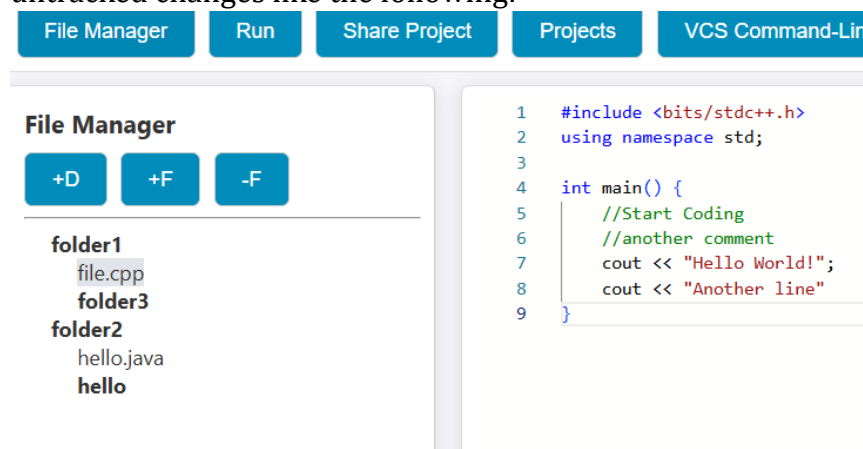
- **Delete the .vcs directory:**

Which will just delete this .vcs directory, and the project is no longer tracked by the .VCS

- **Status:**

Which will track the changes if there are **no changes**, or there are untracked changes, or there are tracked changes.

When adding any changes to the project, these changes will be put into the untracked changes like the following:



If we execute “**vcs status**”:

VCS Command-Line:

```
vcs status
```

```
Untracked changes:
- file.cpp
- hello
```

Because we’ve created another folder, and updated the **file.cpp**, we will continue talking about the status in the followed features.

- **add:**

When executing the ‘**vcs add <Files>**’ we will write the files paths that have been changed, but we still can do ‘vcs add .’ that we all use to add all the changes like the following:

VCS Command-Line:

```
vcs add .
```

```
Traking changes:
- file.cpp
- hello
```

Now when executing the command ‘**vcs status**’:

VCS Command-Line:

```
Type a command and press Enter
```

```
Tracked changes:
- file.cpp
- hello
```

As we can see the changes have been **tracked**.

- **Commit:**

Now we can commit the **tracked changes**:

VCS Command-Line:

```
vcs commit -m "adding new features"
```

```
Committing changes:
- file.cpp
- hello
```

And if we execute '**vcs status**':

VCS Command-Line:

```
vcs status
```

```
No changes, working on clean tree
```

We see that there are no changes left after **committing** all the changes, but still, we can also add and commit only selected changes, just like git.

- **Log:**

Now after **committing** changes, we have several commits (currently two commits), we can log these commits by:

VCS Command-Line:

```
vcs log
```

```
Commit Id:      1%1729922221%1054164371
Author   :      ahmad ahmad@gmail.com
Date      :      2024-10-26T05:57:01.480177400Z
Message: Initial commit

Commit Id:      1%1729923217%-1284588132
Author   :      ahmad ahmad@gmail.com
Date      :      2024-10-26T06:13:37.516078900Z
Message: \"adding new features\"
```

Which will show the commit id, the author, the date, and the message for the commit

- **Revert:**

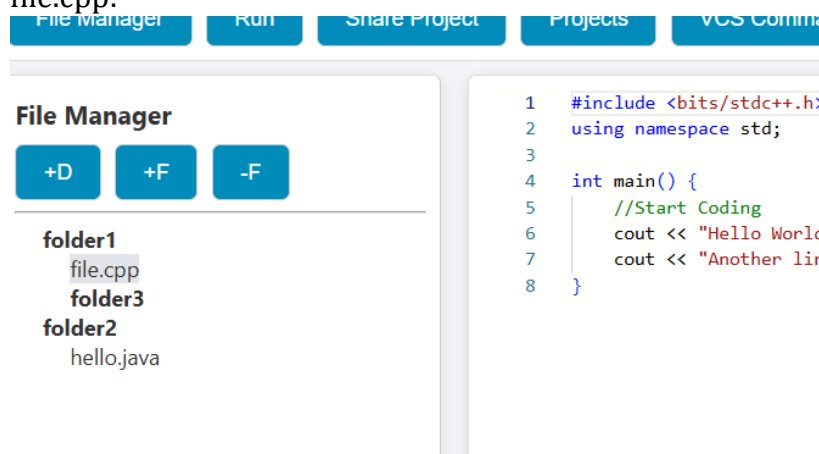
We have commits, so we can revert to some commit by '**vcs revert**' as the following:

```
VCS Command-Line:

vcs revert 1%1729922221%1054164371|

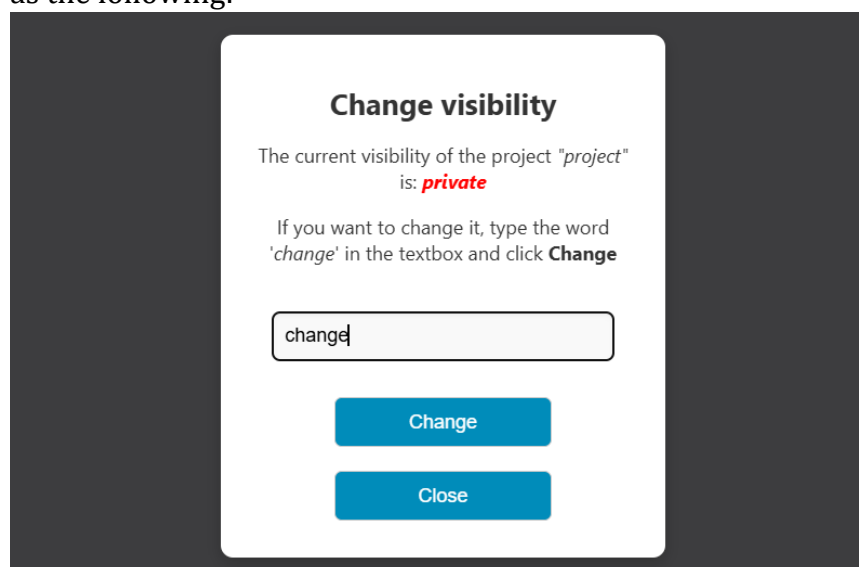
Reverted to commit with id 1%1729922221%1054164371 successfully
```

And the current state of the project is back with no folder created and no update on file.cpp:



- **Fork:**

To fork a project, we have **two choices**, one is to fetch a project from the **public repo** as the following:



First we will share the project on the public repo by **change the visibility** of project, as shown in the above image.

Then we will go to the public repos:

Browse editor

Write the editor email or name

browse

All accounts

Name : ahmad
Email : ahmad@gmail.com

Name : sami
Email : sami@gmail.com

When clicking on ahmad:

All accounts

Name : ahmad
Email : ahmad@gmail.com

Name : sami
Email : sami@gmail.com

repos

Name : project

And when clicking on **'project'**:

TwinCode

Share Project

Proj

FileRun

File Manager

▼ folder1

file.cpp

▶ folder3

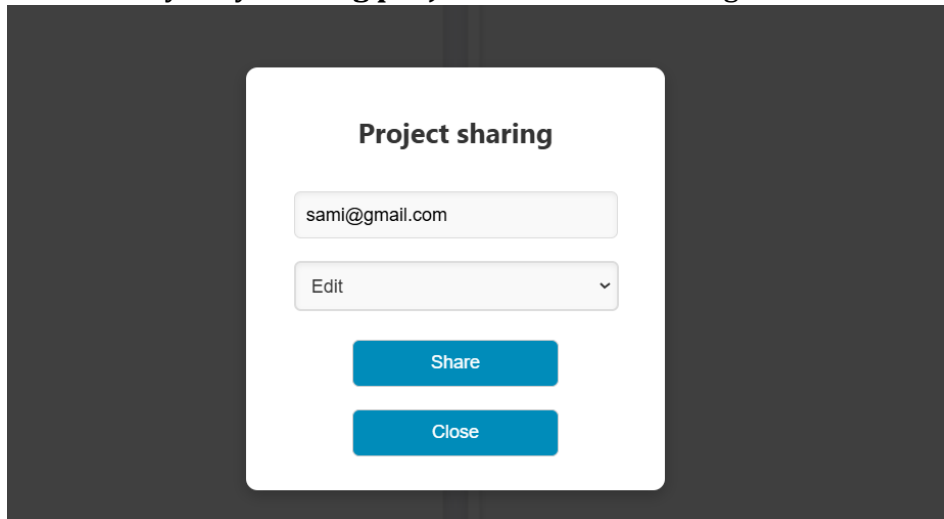
folder2

Choose a file to edit

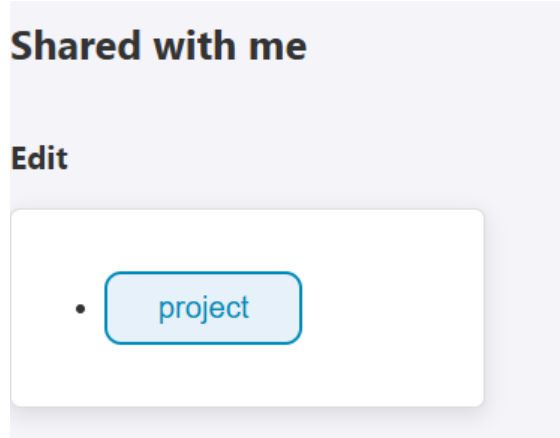
Output:

We can **fork** it from here.

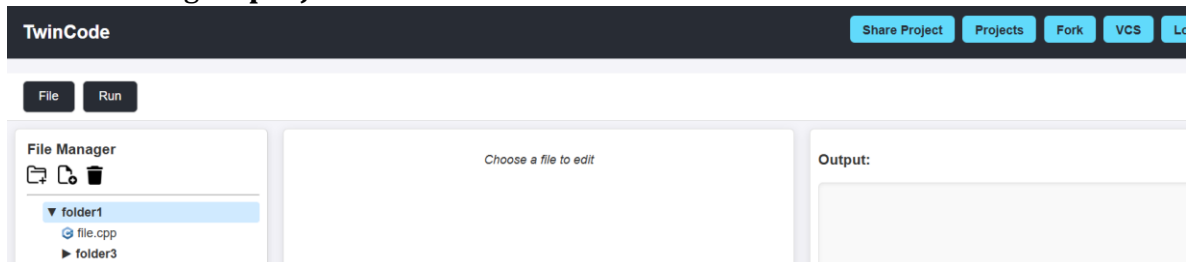
Another way is by **sharing projects** like the following:



Then the project will appear in **Sami's projects**:



When clicking on **project**:



We can fork it also from here.

Means any editor that has the project from the public repo, or has the project from shared with view, he/she can fork it and has his/her own copy of the project without affecting the main project.

- **Branches:**

Let us see how we can create, delete, switch branches:

- **Create:**

VCS Command-Line:

```
vcs branch|
```

```
* main
```

running 'vcs branch <branchName>':

VCS Command-Line:

```
vcs branch new-branch|
```

```
Created a new branch new-branch
```

A new branch have been created, or we can use the **checkout -b** which we will see later.

- **delete:**

VCS Command-Line:

```
vcs branch|
```

```
* main  
new-branch
```

Deleting the branch new-branch:

VCS Command-Line:

```
vcs branch -d new-branch|
```

```
Deleted branch new-branch
```

- **Checkout:**

Let us create a new branch

VCS Command-Line:

```
vcs branch feature|
```

```
Created a new branch feature
```

VCS Command-Line:

```
Type a command and press Enter
```

```
* main  
feature
```

And we can switch to it using:

VCS Command-Line:

```
vcs checkout feature|
```

```
Switched to branch: feature
```

VCS Command-Line:

```
vcs branch|
```

```
* feature  
main
```

Also, we can create a new branch and switch to it at the same time using **vcs checkout -b <branch-name>**, like the following:

VCS Command-Line:

```
vcs checkout -b hotfix
```

```
Created the branch: hotfix and switched to it
```

Executing **vcs branch**:

VCS Command-Line:

```
vcs branch|
```

```
* hotfix  
feature  
main
```

And with this, we have almost implemented all the necessary commands to obtain the git functionality.

Security

Next, we will see what I've implemented in Spring Security:

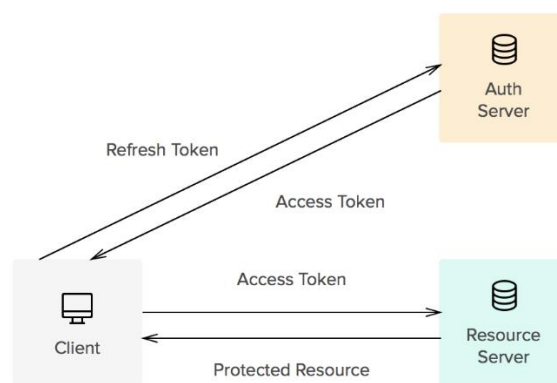
- **Access and refresh tokens:**

```
public AuthenticationResponse register(RegisterDTO request, String role) {
    Client client = Client
        .builder()
        .name(request.getName())
        .email(request.getEmail())
        .password(passwordEncoder.encode(request.getPassword()))
        .role(Role.valueOf(role))
        .build();

    Client savedClient = clientRepository.save(client);
    if (Role.valueOf(role) == Role.EDITOR){
        clientService.addClient(client);
    }
    String accessToken = jwtService.generateAccessToken(client);
    String refreshToken = jwtService.generateRefreshToken(client);
    saveClientToken(savedClient, accessToken);
    return new AuthenticationResponse(accessToken, refreshToken);
}
```

Using this method, which will return an access token for 15 minutes and a refresh token for one week.

And the idea is to give the user a **short life** access token to the system so that he/she can use the services, but to avoid the **interactions between the system** and the **user**, we will generate a refresh token that will have a **long life** (one week in our case) that will generate a new access token when expired.



And the reason we are not just sending a refresh token that will not expire in short is to prevent the **token theft**.

- **Role-based access control (RBAC):**

```
ADMIN(
    Set.of(
        ADMIN_READ,
        ADMIN_CREATE,
        ADMIN_UPDATE,
        ADMIN_DELETE,
        EDITOR_READ,
        EDITOR_CREATE,
        EDITOR_UPDATE,
        EDITOR_DELETE
    )
),
EDITOR(
    Set.of(
        EDITOR_READ,
        EDITOR_CREATE,
        EDITOR_UPDATE,
        EDITOR_DELETE
    )
),
VIEWER(
    Collections.emptySet()
);

private final Set<Permission> permissions;
```

Using this role enum. And the Annotation `@PreAuthorize()`

So for example:

```
@PreAuthorize("hasRole('ADMIN')")
public class AdminController {
    @Autowired
    private AdminService adminService;

    @GetMapping("/get-editors")
    public List<Client> getAllEditors() { return adminService.getAllEditors(); }

    @DeleteMapping("/remove-editor/{editorId}")
    public void removeEditor(@PathVariable Long editorId) { adminService.deleteEditor(editorId); }

    @GetMapping("/get-editor-projects/{editorId}")
    public List<Project> getEditorProjects(@PathVariable Long editorId) { ... }
```

When any role other than the admin accesses this controller, it will return a forbidden, so that no clients can use the admin services unless the admin.

For the view role, currently and in our case it has to permission to call any api from the server, all we will do is to check the view credentials from the filesystem.

Clean Code Principles (Uncle Bob)

We will discuss the **Robert C. Martin** clean code principles, which are:

- **Naming:**

In my code, I haven't named a class with a general (not specific) name, every class name has a specific name related to the functionality of this class.

- **Null:**

I haven't returned **NULL** in a method because of the "NullPointerException" also will increase the "CYC", instead, when I'm forced to return null, I will throw an exception, and in that way, we will understand the error instead of looking for it when returning null.

- **Parameters:**

In my code, almost all the methods don't take more than 4 parameters, the methods often take 2-3 parameters or at most 4, but for some methods I'm passing up to 5 parameters.

- **Flag Arguments:**

Also, no flag arguments (boolean value) will be passed to any method, instead, I've separated the methods that takes a flag argument into two methods.

Effective Java Items

Here I will discuss some of the Effective Java Items in my code with some examples for each one:

- **Item 1:**

Consider static factory methods instead of constructors.

Spring is already injecting and creating the singleton objects when use **@Autowired**

```
@Service 3 usages  ⚡ AhmdJabr
public class FileItemService {
    @Autowired
    private FileItemRepository fileItemRepository;

    public FileItem getFileById(Long id){ 5 usages  ⚡ AhmdJabr
        return fileItemRepository.findById(id).orElseThrow(
            () -> new RuntimeException("File with id " + id + " not found")
        );
    }

    public FileItem createFile(FileItem fileItem) { return fileItemRepository.save(fi
```

- **Item 2:**

Consider a builder when faced with constructors that have many parameters.

Classes like **Comment**, **Project**, **ChangeHolder** will use the Builder pattern like the following:

```
    }  
    }  
    ChangeHolder change = ChangeHolder.builder()  
        .change(changeType)  
        .fileType(fileType)  
        .content(content)  
        .build();  
    changes.put(fileItem.getId(), change);  
  
    tracked.remove(fileItem.getId());
```

- **Item 3:**

Enforce the singleton property with a private constructor or an enum type.

I've talked about this in the first item that I'm using **@Autowired** for the services and the repositories and the utilities, and so, spring it self will handle the singleton creation.

- **Item 4:**

Enforce noninstantiability with a private constructor.

Which is good for Utility classes, so I've provided a private constructor for all the utility classes to prevent instantiating an object form these classes:

```
private FileUtil(){ new *  
    throw new IllegalStateException("Utility class cannot be instantiated");  
}
```

With this, we can't create an object from this class.

- **Item 6:**

Avoid creating unnecessary objects.

In our case, Instead of initializing Integer (type-wrapper), we will use the primitive types when we can use them, also we will use the StringBuilder instead of the String class when appending to a string.

- **Item 7:**

Eliminate obsolete object references.

Because the garbage collector will not collect these objects, and I've tried as possible not to have such references that will not be collected.

- **Item 9:**

Prefer try-with-resources to try-finally.

Because this approach will just create objects and delete them once done, like the following in my code:

```
try (FileOutputStream fileOutputStream = new FileOutputStream(file);
    ObjectOutputStream objectOutputStream = new ObjectOutputStream(fileOutputStream)){
    objectOutputStream.writeObject(projectStructure);
    System.out.println("Object written to " + file.getAbsolutePath());
} catch (IOException e) {
    throw new RuntimeException(e);
}
```

Here, the **fileOutputStream** will be deleted once we exit the try block.

- **Item 10:**

Obey the general contract when overriding equals.

I've included the equals method in both Comment and Project and other classes, here is the Comment class with overriding the equals method:

```
@Override new *
public boolean equals(Object object) {
    if (this == object) return true;
    if (object == null || getClass() != object.getClass()) return false;
    Comment comment = (Comment) object;
    return Objects.equals(editorName, comment.editorName)
        && Objects.equals(editorEmail, comment.editorEmail)
        && Objects.equals(start, comment.start)
        && Objects.equals(end, comment.end)
        && Objects.equals(date, comment.date)
        && Objects.equals(content, comment.content);
}
```

- **Item 12:**

Always Override `toString()` method.

In my code, any class that has at least one attribute, will have the method `toString()`.

Here is the `toString()` method in the Client class:

```
@Override  @ AhmdJabr
public String toString() {
    return "Client{" +
        "id=" + id +
        ", name='" + name + '\'' +
        ", email='" + email + '\'' +
        ", role=" + role +
        '}';
}
```

Which prints the client's id, name, email, role.

SOLID Principles

Next, we will talk about the SOLID Principles:

- **SRP (Single Responsibility Principle):**

A Class should have only one reason to change (it should have only one responsibility).

And that's what I've implemented in my code, each class has one responsibility, all the methods and attributes are related to the class (*high cohesion*), which will make the classes more maintainable.

For example, The **VCSStorageService** will have all the methods that are related to the version control system, making it more maintainable:

```
public void initVCS(Long projectId) {...}

public void deleteVCS(Long projectId){...}

public Map<String, List<String>> status(Long projectId) {...}

@Transactional  @ AhmdJabr *
public List<String> add(Long projectId, List<String> files) throws IOException {...}

@Transactional  1 usage  @ AhmdJabr *
public String log(Long projectId) {...}
```


- **DIP (Dependency Inversion Principle):**

High level modules shouldn't depend on low level modules, both should depend on abstractions.

In Spring, the **@Autowired** annotation enables **Dependency Inversion** by allowing a high-level class to depend on an **abstraction** (interface) rather than a specific implementation. Instead of the class creating the dependency itself, Spring injects the required dependency at runtime, and in my code, all the services and the repositories and the utilities are injected using the **@Autowired**, for example in the controllers:

```
public class AdminController {  
    @Autowired  
    private AdminService adminService;  
  
    @GetMapping("/get-editors")  
    public List<Client> getAllEditors() { return adminService
```

And in the services:

```
@RequiredArgsConstructor  
public class LogoutService implements LogoutHandler {  
    @Autowired  
    private final TokenRepository tokenRepository;  
  
    @Override  
    public void logout(HttpServletRequest request, HttpServletResponse  
        final String authHeader = request.getHeader("Authorization")
```

In the utilities:

```
@Service  
public class FileUtil {  
    @Autowired  
    private EncryptionUtil encryptionUtil;  
  
    private FileUtil() {  
        throw new IllegalStateException("Utility class cannot be instantiated");  
    }  
  
    public void createFolder(String folderPath) {  
        System.out.println(folderPath);  
        File folder = new File(folderPath);  
        if (!folder.exists()) {
```

And we are ensuring that everything depends on the **abstraction**.

Last thing is, in such systems, there are no complicated relations between objects, because the system doesn't contain that number of relations between them, so I've just considered the first and the last principles of the SOLID principles, and for the design patterns, I've already talked about the singleton and the builder patterns.

Thanks

Ahmad Nabeel Jaber